i-SOFTZONE Employee Management System

Full-Stack Web Application

Developer: Rishi Dhakad

GitHub: @rishixh0520

1. Introduction

Objective

To build a comprehensive, cloud-deployed Employee Management System that streamlines human resource operations for organizations.

Scope

- Centralized employee lifecycle management
- Advanced leave workflow with multi-level approvals
- Automated payroll processing
- Attendance and asset tracking
- Insightful analytics dashboards for executives and managers

2. Architecture

A modern, scalable full-stack architecture using the MERN/PERN stack.

- **Frontend**: React (Vite Build) deployed as a SPA on **Vercel**.
- **Backend**: Node.js & Express API deployed on **Render**.
- **Database**: Cloud-hosted **Neon PostgreSQL**.
- **Communication**: RESTful API over HTTPS.

Secure, fast, and globally accessible.

3. Database Design

Relational database architecture designed for data integrity.

- **19 Tables, 1 View, 1 Stored Function**
- **Core Entities**: Users, Employee Profiles, Departments, Skills
- **Operational Entities**: Leaves, Attendance, Salaries, Assets
- **Security & Tracking**: Audit Logs, Refresh Tokens, Password Resets

Robust foreign key constraints ensure complete relational mapping.

4. Key Features (Part 1)

Authentication & Roles

Secure JWT-based auth (Access + Refresh tokens). Role-based access control (Admin, Manager, HR, Employee).

Employee & Master Data

Complete CRUD operations for employees, departments, and skills. Advanced search and profile image uploads.

Leave Management

Customizable leave types (Casual, Sick, Earned, Maternity). Hierarchical approval workflow (Employee → Manager → HR).

5. Key Features (Part 2)

Payroll Processing

Automated salary calculation including Basic, HRA, DA, and deductions (TDS, ESI, PF). Payslip generation.

Attendance & Assets

Daily clock-in/out tracking and comprehensive asset registry with employee allocation tracking.

Analytics & Reports

Executive dashboards, department metrics, and manager insights. Export reports to PDF, CSV, and Excel.

6. Technical Challenges

- **Complex Authorization Flow**: Implementing multi-level hierarchical leave approvals and ensuring users only access their permitted data.
- **Data Integrity**: Managing the complex relationships in PostgreSQL (e.g., maintaining accurate leave balances safely).
- **Security**: Implementing robust JWT token rotation, securing password resets, and defending against common vulnerabilities.
- **Report Generation**: Ensuring reliable and formatted exports across PDF and Excel formats.

7. Deployment

Fully deployed to the cloud with continuous integration in mind.

- **Frontend Platform**: Vercel (`https://login-app-e4zw.vercel.app`)
- **Backend API**: Render (`https://login-app-latest-kmpt.onrender.com`)
- **Database**: Neon (Serverless Postgres)
- **API Documentation**: Swagger UI integrated into the backend

Environment variables isolate configuration across local, staging, and production environments.

8. Learning Outcome

Building this system provided deep, practical experience in:

- Designing and optimizing a complex relational database schema.
- Implementing secure authentication and advanced role-based access control.
- Architecting a scalable backend with Node.js and Express.
- Building dynamic, responsive user interfaces with React.
- Deploying a multi-tier architecture to cloud platforms (Vercel, Render, Neon).

Thank You!

Questions & Answers

GitHub: @rishixh0520

Email: rishixh0520@gmail.com